

# **Programming Language Project**

by

Aye Khin Khin Hpone (Yolanda Lim), st125970

Applegate Tun Oo, st126690

Win Htut Naing, st126687

AT70.07 — Programming Languages and Compilers  
Assignment 2

Instructor: Prof. Phan Minh Dung

Teaching Assistant: Akaradet Sinsamersuk

Asian Institute of Technology  
School of Engineering and Technology  
Thailand  
April 2026

## **ABSTRACT**

This report presents the design and implementation of a statically-typed programming language developed for a programming languages and compilers assignment. The language supports four primitive types — Integer, Float, Boolean, and String — with type inference, arithmetic and boolean expressions, assignment statements, if-then-else and while-loop control flow, function abstraction with value parameter passing, and a built-in `print()` function. The implementation follows a classical compiler pipeline consisting of lexical analysis, recursive-descent parsing, abstract syntax tree construction, static type checking, and tree-walking interpretation. A command-line runner and a desktop user interface are provided so that script files can be loaded, executed, and inspected. The report describes the language grammar, the type inference algorithm, the implementation structure, and the testing used to validate the system.

# CONTENTS

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>ABSTRACT</b>                                     | <b>ii</b> |
| <b>1 INTRODUCTION</b>                               | <b>1</b>  |
| 1.1 Background and Motivation . . . . .             | 1         |
| 1.2 Language Overview . . . . .                     | 1         |
| 1.3 Report Structure . . . . .                      | 2         |
| <b>2 LANGUAGE DESIGN AND GRAMMAR</b>                | <b>3</b>  |
| 2.1 Types . . . . .                                 | 3         |
| 2.1.1 Integer . . . . .                             | 3         |
| 2.1.2 Float . . . . .                               | 3         |
| 2.1.3 Boolean . . . . .                             | 3         |
| 2.1.4 String . . . . .                              | 3         |
| 2.1.5 Void . . . . .                                | 3         |
| 2.2 Static Typing . . . . .                         | 4         |
| 2.3 Token Specification . . . . .                   | 4         |
| 2.4 Context-Free Grammar . . . . .                  | 4         |
| 2.4.1 Program Structure . . . . .                   | 4         |
| 2.4.2 Arithmetic Expressions . . . . .              | 5         |
| 2.4.3 Boolean Expressions . . . . .                 | 5         |
| 2.4.4 Statements . . . . .                          | 5         |
| 2.4.5 Function Definitions and Calls . . . . .      | 5         |
| 2.5 Operator Precedence and Associativity . . . . . | 5         |
| <b>3 TYPE INFERENCE ALGORITHM</b>                   | <b>7</b>  |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 3.1      | Overview . . . . .                                 | 7         |
| 3.2      | Type Environment . . . . .                         | 7         |
| 3.3      | Inference Rules . . . . .                          | 7         |
| 3.3.1    | Literals . . . . .                                 | 7         |
| 3.3.2    | Arithmetic Expressions . . . . .                   | 8         |
| 3.3.3    | Boolean Expressions . . . . .                      | 8         |
| 3.3.4    | Assignment Statement . . . . .                     | 8         |
| 3.3.5    | If-Then-Else . . . . .                             | 8         |
| 3.3.6    | While-Loop . . . . .                               | 9         |
| 3.3.7    | Function Definition and Call . . . . .             | 9         |
| 3.4      | Type Error Reporting . . . . .                     | 9         |
| 3.5      | Example Type Inference Traces . . . . .            | 9         |
| <b>4</b> | <b>SYSTEM ARCHITECTURE AND IMPLEMENTATION</b>      | <b>11</b> |
| 4.1      | Compiler Pipeline . . . . .                        | 11        |
| 4.2      | Project Structure . . . . .                        | 11        |
| 4.3      | Technology Stack . . . . .                         | 11        |
| 4.4      | Lexical Analyser . . . . .                         | 12        |
| 4.4.1    | Implemented Lexer and Symbol Table Scope . . . . . | 12        |
| 4.5      | Symbol Table . . . . .                             | 13        |
| 4.6      | Parser . . . . .                                   | 14        |
| 4.7      | Abstract Syntax Tree . . . . .                     | 14        |
| 4.8      | Type Checker . . . . .                             | 14        |
| 4.8.1    | Implemented Type System Scope . . . . .            | 14        |
| 4.9      | Interpreter . . . . .                              | 15        |
| 4.9.1    | Implemented Interpreter Scope . . . . .            | 15        |
| 4.10     | Graphical User Interface . . . . .                 | 15        |

|          |                                                   |           |
|----------|---------------------------------------------------|-----------|
| <b>5</b> | <b>TESTING AND VERIFICATION</b>                   | <b>16</b> |
| 5.1      | Test Cases . . . . .                              | 16        |
| 5.1.1    | Lexer and Symbol Table . . . . .                  | 16        |
| 5.1.2    | Arithmetic Expressions . . . . .                  | 17        |
| 5.1.3    | Boolean Expressions . . . . .                     | 17        |
| 5.1.4    | Assignment Statements . . . . .                   | 17        |
| 5.1.5    | If-Then-Else . . . . .                            | 17        |
| 5.1.6    | While-Loop . . . . .                              | 17        |
| 5.1.7    | Functions . . . . .                               | 17        |
| 5.1.8    | Print . . . . .                                   | 18        |
| 5.2      | Type Error Detection . . . . .                    | 18        |
| 5.3      | Automated Test Suite . . . . .                    | 18        |
| 5.4      | Error Handling . . . . .                          | 18        |
| <b>6</b> | <b>CONCLUSION</b>                                 | <b>19</b> |
|          | <b>REFERENCES</b>                                 | <b>20</b> |
|          | <b>APPENDIX A: SOURCE CODE AVAILABILITY</b>       | <b>21</b> |
|          | <b>APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS</b> | <b>22</b> |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background and Motivation

Programming languages provide the formal structure through which algorithms are expressed and executed. A compiler or language processor is responsible for turning source code into a form that can be analysed or executed while also detecting errors as early as possible. Core compiler concepts such as lexical analysis, parsing, abstract syntax trees, symbol tables, static type checking, and interpretation are therefore fundamental topics in programming language education.

The motivation for this project is to apply those concepts in a compact but complete language implementation. By building a small statically-typed language, the project demonstrates how syntax, semantics, and execution can be connected in a coherent system. The assignment also provides an opportunity to study how type inference can improve usability while still preserving compile-time error checking.

### 1.2 Language Overview

The implemented language supports four primitive types: Integer, Float, Boolean, and String. It uses static typing with inference, so programmers do not need to write explicit type declarations. Expressions include arithmetic operations with standard precedence and Boolean comparisons through equality and inequality. The statement-level constructs include assignment, if-then-else, while-loop, function definition, function call, return, and the built-in `print()` function.

Programs are processed through a lexer, a recursive-descent parser, a type checker, and an interpreter. The final system can be executed from the command line or through a desktop UI that allows the user to enter a script, run it, and inspect the resulting tokens, AST, inferred types, and output.

### **1.3 Report Structure**

The remainder of this report is organised as follows. Chapter 2 presents the language design, token specification, grammar, and operator precedence rules. Chapter 3 explains the type inference algorithm and the semantic rules used by the checker. Chapter 4 describes the overall system architecture and the main implementation components. Chapter 5 discusses testing and verification, including the automated regression tests. Chapter 6 concludes the report and summarises the final outcomes of the project.

## CHAPTER 2

### LANGUAGE DESIGN AND GRAMMAR

#### 2.1 Types

The language supports four primitive types. These types are sufficient to cover the assignment requirements while keeping the semantics simple and predictable.

##### 2.1.1 *Integer*

The Integer type represents whole numbers such as 0, 7, and 42. Integer literals are written as one or more decimal digits without quotation marks or a decimal point.

##### 2.1.2 *Float*

The Float type represents real-valued numbers written with a decimal point, such as 3.14 or 20.5. Float values are used in arithmetic expressions and may also arise through numeric promotion.

##### 2.1.3 *Boolean*

The Boolean type represents logical truth values. The two Boolean literals are `true` and `false`. Boolean values are used in conditions and as results of comparison expressions.

##### 2.1.4 *String*

The String type represents sequences of characters enclosed in double quotes, for example `"hello"`. Strings can be assigned to variables and printed through the built-in `print()` function. The surrounding quotes are stripped by the parser when the `Literal` node is created, so the value stored internally and printed at runtime is the bare character sequence without quotes.

##### 2.1.5 *Void*

`Void` is an internal type used by the type system to represent the return type of functions that contain no return statement. It is not a type that programmers write explicitly; it exists solely inside the `LanguageType` enum so that the symbol table can record a well-defined type for

every function definition, including those that produce no value.

## 2.2 Static Typing

The language uses static typing, which means type errors are detected before execution rather than during program execution. No explicit variable type declarations are required. Instead, the type checker infers types from literals, assignments, operations, function calls, and return statements. Once a variable type is established, later assignments must remain consistent with that type.

## 2.3 Token Specification

Table 2.1 summarises the major token classes used by the language.

Table 2.1. Token Specification

| Token Category       | Example Lexemes                            |
|----------------------|--------------------------------------------|
| INT_LITERAL          | 0, 10, 42                                  |
| FLOAT_LITERAL        | 3.14, 20.5                                 |
| BOOL_LITERAL         | true, false                                |
| STRING_LITERAL       | "hello"                                    |
| IDENTIFIER           | variable and function names such as x, add |
| Arithmetic operators | +, -, *, /                                 |
| Comparison operators | ==, !=                                     |
| Assignment operator  | =                                          |
| Keywords             | if, else, while, def, return, print        |
| Delimiters           | ( ) { } , ;                                |

## 2.4 Context-Free Grammar

The language grammar can be described by the context-free grammar  $G = (V_N, V_T, P, S)$ , where  $V_N$  is the set of non-terminals,  $V_T$  is the set of terminals,  $P$  is the set of production rules, and  $S$  is the start symbol program. The grammar is implemented through recursive-descent parsing functions.

### 2.4.1 Program Structure

```
1 program    -> statement* EOF
2 statement -> assignment ';'
3           | if_stmt
4           | while_stmt
5           | func_def
6           | print_stmt ';' ;
```

```

7 | return_stmt ';'
8 | expr ';'

```

## 2.4.2 Arithmetic Expressions

```

1 expr  -> term (('+' | '-') term)*
2 term  -> factor (('*' | '/') factor)*
3 factor -> INT_LITERAL
4       | FLOAT_LITERAL
5       | STRING_LITERAL
6       | BOOL_LITERAL
7       | IDENTIFIER
8       | IDENTIFIER '(' args? ')',
9       | '(' expr ')',

```

## 2.4.3 Boolean Expressions

```

1 bool_expr -> expr '==' expr
2           | expr '!=' expr
3           | BOOL_LITERAL

```

In the implementation, if `bool_expr` parses an expression and finds no following `==` or `!=` operator, the expression is returned as-is. The type checker then verifies that its inferred type is Boolean. This allows the grammar to remain simple while keeping the parser deterministic.

## 2.4.4 Statements

```

1 assignment -> IDENTIFIER '=' expr
2 if_stmt    -> 'if' '(' bool_expr ')' block ('else' block)?
3 while_stmt -> 'while' '(' bool_expr ')' block
4 print_stmt -> 'print' '(' expr ')',
5 return_stmt -> 'return' expr
6 block      -> '{' statement* '}',

```

## 2.4.5 Function Definitions and Calls

```

1 func_def -> 'def' IDENTIFIER '(' params? ')', block
2 params  -> IDENTIFIER (',' IDENTIFIER)*
3 args    -> expr (',' expr)*

```

Function calls use value parameter passing. This means the argument values are evaluated before the call and copied into the function's local environment.

## 2.5 Operator Precedence and Associativity

Operator precedence is encoded structurally in the recursive-descent parser. Comparison is lowest, addition and subtraction are next, and multiplication and division have the highest precedence among binary operators. Arithmetic operators are left-associative.

Table 2.2. Operator Precedence and Associativity

| Level | Operators | Associativity    |
|-------|-----------|------------------|
| 1     | ==, !=    | comparison level |
| 2     | +, -      | left-associative |
| 3     | *, /      | left-associative |

## CHAPTER 3

### TYPE INFERENCE ALGORITHM

#### 3.1 Overview

Type inference in this project means that the programmer does not write explicit type declarations for variables or functions. Instead, the compiler infers types from assignments, literals, operations, function calls, and return statements. This keeps the language simple while still preserving static type safety.

The type checker runs after parsing. It receives the AST and validates the program before execution. Any inconsistent use of types is reported as a type error, and execution is stopped.

The interpreter stage does not repeat this validation: it walks the same AST with ordinary Python representations of values. Static type information remains in the symbol table produced by the checker; run-time values are not separately labelled with `LanguageType`, except that `print()` formats output by inspecting the value's Python type.

#### 3.2 Type Environment

The type environment is represented by the symbol table. Conceptually, it is a mapping

$$\Gamma : Identifier \rightarrow Type$$

The environment grows as the checker visits the AST. On first assignment, a variable name is inserted with its inferred type. When a new block or a function call is entered, a child environment is created so that nested scopes can be checked independently while still accessing outer bindings when needed.

#### 3.3 Inference Rules

##### 3.3.1 Literals

Literal nodes have direct types:

$$\begin{aligned}\Gamma \vdash 42 &: \text{Integer} \\ \Gamma \vdash 3.14 &: \text{Float} \\ \Gamma \vdash \text{true} &: \text{Boolean} \\ \Gamma \vdash \text{'hi'} &: \text{String}\end{aligned}$$

### 3.3.2 *Arithmetic Expressions*

The arithmetic operators `+`, `-`, `*`, and `/` accept only numeric operands. If both operands are `Integer`, the result is `Integer`, except for division, which produces `Float`. If at least one operand is `Float`, the result is `Float`. Any non-numeric operand causes a type error.

### 3.3.3 *Boolean Expressions*

The comparison operators `==` and `!=` always produce `Boolean`. Numeric comparison is allowed for arithmetic expressions. The current implementation also permits Boolean-to-Boolean comparison because the sample programs use `flag!=false`. Comparisons involving `String` operands are rejected (so string equality is not part of the type system). Other combinations are rejected.

### 3.3.4 *Assignment Statement*

Assignments define variable types on first use. If a variable does not yet exist in the type environment, the checker infers the type of the right-hand side and inserts the variable into the environment. If the variable already exists, the new value must have the same type.

$$\text{If } x \notin \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma, x : T \vdash x = e$$

$$\text{If } x : T \in \Gamma \text{ and } \Gamma \vdash e : T, \text{ then } \Gamma \vdash x = e$$

### 3.3.5 *If-Then-Else*

The condition of an `if` statement must be `Boolean`. The then-block and else-block are checked in child scopes so that local operations can be validated cleanly.

### 3.3.6 While-Loop

The condition of a `while` loop must also be Boolean. The loop body is checked in a child scope and must not violate the types of variables already introduced in the surrounding environment.

### 3.3.7 Function Definition and Call

Function parameter types are inferred from the first call to the function. After that first call, subsequent calls must use arguments of the same types. The return type is inferred from the return statements in the function body. If the function returns inconsistent types, the checker reports an error.

Recursive calls are intentionally rejected in this implementation because they were not required by the assignment and would require additional inference and control-flow analysis.

## 3.4 Type Error Reporting

Type errors are reported with source positions so that the user can locate the problem quickly. The format used by the system is:

```
[line X, col Y] TypeError: message
```

Examples include assigning a `String` to an `Integer` variable, using a non-Boolean condition in an `if` statement, or supplying the wrong argument type to a function.

## 3.5 Example Type Inference Traces

Consider the following program fragment:

```
1 x = 10;  
2 y = 20.5;  
3 z = x + y;  
4 print(z);
```

The checker infers:

- `x`: `Integer`
- `y`: `Float`
- `x + y`: `Float`

- `z: Float`

For a function example:

```
1 def add(a, b) {  
2     return a + b;  
3 }  
4  
5 result = add(2, 3.5);
```

The first function call infers `a: Integer` and `b: Float`. The expression `a + b` is therefore `Float`, so the function return type is inferred as `Float`. The variable `result` is also inferred as `Float`.

## CHAPTER 4

### SYSTEM ARCHITECTURE AND IMPLEMENTATION

#### 4.1 Compiler Pipeline

The implemented system follows a four-stage pipeline. Source code is first converted into tokens by the lexer. The parser then constructs an AST from the token stream. After parsing, the type checker validates and infers types across the AST. Finally, the interpreter executes the checked program and emits output line by line.

The full processing pipeline is:

Source Code → Tokens → AST → Type Check → Execution Output

This design keeps syntax handling independent from semantic checking and execution, which simplifies testing and debugging.

#### 4.2 Project Structure

The main source files and their responsibilities are summarised below.

Table 4.1. Main Source Files

| File                       | Responsibility                                       |
|----------------------------|------------------------------------------------------|
| main.py                    | Command-line runner for sample code or a script file |
| ui.py                      | Desktop UI for editing and running a script          |
| components/tokens.py       | Token classes and token categories                   |
| components/lexica.py       | Lexical analysis                                     |
| components/ast_nodes.py    | AST node definitions                                 |
| components/parser.py       | Recursive-descent parser                             |
| components/ast_printer.py  | AST renderer for debugging/reporting                 |
| components/type_checker.py | Static type checker and inference                    |
| components/interpreter.py  | Tree-walking interpreter                             |
| components/pipeline.py     | Shared pipeline for CLI, UI, and tests               |

#### 4.3 Technology Stack

The project uses Python and standard-library tools only.

Table 4.2. Technology Stack

| Tool                     | Purpose                           |
|--------------------------|-----------------------------------|
| Python 3                 | Core implementation language      |
| <code>dataclasses</code> | AST and runtime helper structures |
| <code>tkinter</code>     | Desktop graphical interface       |
| <code>unittest</code>    | Automated testing                 |

## 4.4 Lexical Analyser

The lexer reads the source character by character and emits tokens for identifiers, literals, keywords, operators, and delimiters. It preserves line and column information in each token to support precise diagnostics. Invalid characters and unterminated strings are reported immediately with source positions.

Two-character operators (`==` and `!=`) are resolved with a single lookahead step before single-character operators are matched. Identifiers are scanned greedily and then checked against the `KEYWORDS` dictionary; unrecognised names become `IDENTIFIER` tokens. Number scanning detects the optional decimal point and lookahead digit that distinguish a `FLOAT_LITERAL` from an `INT_LITERAL`.

### 4.4.1 *Implemented Lexer and Symbol Table Scope*

The lexer and symbol table work covers the full foundational scope assigned for this component. Concretely, this stage produces the token stream consumed by the parser and provides the symbol storage consulted by the type checker:

- `TokenType` enum covering all four literal types, `IDENTIFIER`, six keywords (`if`, `else`, `while`, `def`, `return`, `print`), four arithmetic operators, two comparison operators, the assignment operator, six delimiters, and `EOF`,
- frozen `Token` dataclass carrying `token_type`, `lexeme`, `line`, and `column` for precise diagnostics,
- keyword mapping and two-character operator tokenisation in the `Lexer` class with source-positioned error messages,
- identifier scanning with keyword fallback; integer, float, Boolean (`true/false`), and string literal recognition,

- `scoped SymbolTable` with duplicate-symbol detection, parent-chain lookup, and child-scope creation,
- `LanguageType` enum (`Integer`, `Float`, `Boolean`, `String`, `Void`) used to label variables and function signatures,
- `VariableSymbol` and `FunctionSymbol` subclasses storing type, parameter list, and initialisation state,
- `format_table()` method that renders the symbol table for the CLI and UI inferred-types display.

## 4.5 Symbol Table

The symbol table is implemented in `components/symbol_table.py` and serves as the shared store for both the type checker and the formatted output displayed to the user.

**LanguageType enum.** The five language types are defined as an enum: `Integer`, `Float`, `Boolean`, `String`, and `Void`. The first four correspond to the language’s primitive types. `Void` is the inferred return type of functions that contain no return statement, keeping the type system closed under function definitions.

**Symbol class hierarchy.** The base `Symbol` class holds a name and a `symbol_type`. Two subclasses specialise it: `VariableSymbol` adds an `initialized` flag that tracks whether the variable has received a value, and `FunctionSymbol` adds a `parameters` list of (`name`, `LanguageType`) pairs representing the inferred function signature.

**Scoped lookup.** Each `SymbolTable` instance holds a `_symbols` dictionary for its own scope and an optional parent reference. `lookup()` walks the parent chain so that inner scopes can read outer variables. `exists_in_current_scope()` checks only the local dictionary, which `define_variable()` and `define_function()` use to raise a `SymbolTableError` on duplicate definitions. A new child scope is obtained by calling `child_scope()`, which is used by the type checker when entering blocks and function bodies.

**Formatted output.** `format_table()` renders the symbol table as a fixed-width text table with columns for name, kind, type, initialisation status, and parameters. This output appears as Stage 3 in the CLI pipeline and in the inferred-types tab of the desktop UI.

## 4.6 Parser

The parser is implemented as a handwritten recursive-descent parser. It builds AST nodes for assignments, print statements, returns, blocks, conditional statements, while-loops, function definitions, and function calls. Operator precedence is encoded structurally through separate parsing functions for factors, terms, expressions, and boolean expressions.

Distinguishing an assignment statement from a bare expression statement is handled with a one-token lookahead: when the current token is an `IDENTIFIER` and the next token is `ASSIGN` (`=`), the parser takes the assignment path; otherwise it falls through to the expression path. This avoids backtracking and keeps the parser deterministic.

String literal values stored in `Literal` nodes have their surrounding double-quote characters stripped during parsing, so `"hello"` produces a `Literal` whose value is the plain Python string `hello`.

The `_bool_expr()` method handles the condition position in `if` and `while` statements. It first checks for a bare `BOOL_LITERAL` and returns it immediately. Otherwise it parses an expression and looks for a `==` or `!=` operator; if neither is present the expression is returned as-is, and the type checker is responsible for verifying that the result is `Boolean`.

### 4.6.1 *Implemented Parser and AST Scope*

The parser and AST work covers the full syntactic scope assigned for this component. Concretely, this stage transforms the token stream into the tree structure consumed by the type checker and interpreter:

- recursive-descent `Parser` class producing a `Program` root node from a `list[Token]`,
- `Assign` node: variable assignment with a target name and a value sub-tree,
- `BinaryOp` node: both arithmetic (`+`, `-`, `*`, `/`) and comparison (`==`, `!=`) operations encoded in the same node with an `op` string field,
- `Literal` node: integer, float, `Boolean`, and string values (string quotes stripped at parse time),
- `Identifier` node: variable and function name references,
- `If` node: condition, mandatory then-block, and an `Optional[Block]` else-block,

- `While` node: condition and body block,
- `FunctionDef` node: function name, list of parameter *names* only (types are deferred to the type checker), and a body block,
- `FunctionCall` node: callee name and a list of argument sub-trees,
- `Print` node: single expression argument,
- `Return` node: single expression to return,
- `Block` node: brace-delimited sequence of statements used as function bodies, if-branches, and loop bodies,
- `Program` node: top-level list of statements,
- all nodes carry `line` and `column` metadata for source-positioned diagnostics.

## 4.7 Abstract Syntax Tree

The AST is the main intermediate representation shared by the checker and the interpreter. It contains nodes for literals, identifiers, binary operations, assignments, block statements, `if`, `while`, function definitions, function calls, `return`, and the built-in `print()` function. Each node stores `line` and `column` metadata.

**Node design decisions.** `FunctionDef` stores only the parameter names, not their types; type information is not available at parse time and is inferred by the type checker from the first call site. `If.else_block` is `Optional[Block]`: the `else` clause is genuinely absent for single-branch conditionals, and the interpreter checks for `None` rather than inserting an empty block. `BinaryOp` serves dual duty — it represents both arithmetic and comparison operations, distinguished at semantic time by the `op` string (`+/-/*//` versus `==/!=`).

**AST printer.** `components/ast_printer.py` implements an `ASTPrinter` class that walks the AST using a visitor dispatch: for each node it looks up a method named `_visit_<NodeClassName>` and calls it. The output is an indented text tree where each line names the node type and its key fields (name, operator, value, argument count, etc.), indented four spaces per level. This output appears as Stage 2 in the CLI pipeline and in the AST tab of the desktop UI.

## 4.8 Type Checker

The type checker traverses the AST before execution. It uses the symbol table to record variable and function information, infer types on first assignment, validate arithmetic and comparison expressions, enforce Boolean conditions for control flow, infer function parameter types from the first call, and infer function return types from return statements. Programs that fail these rules are rejected before interpretation.

Types are enforced entirely during this pass. The interpreter does not consult the symbol table to validate each operation; it assumes the checked program and uses native Python values.

### 4.8.1 *Implemented Type System Scope*

The implemented type system work covers the full semantic scope assigned for the type system component. In presentation terms, this stage is responsible for turning a syntactically valid AST into a semantically valid program that can be executed with confidence:

- static typing rules for arithmetic, comparison, assignment, control flow, and functions,
- type checking for expressions and statements,
- variable type inference from first assignment,
- function parameter inference from the first call,
- function return type inference from return statements,
- source-positioned type errors when a program violates a typing rule.

## 4.9 Interpreter

The interpreter is a tree-walking evaluator that executes the checked AST. It uses nested runtime environments for variables and function calls. Assignments store values in the current or nearest enclosing scope, `if` executes only the selected branch, and `while` re-evaluates its condition until it becomes false. Function calls create child environments, bind arguments by value, and return through an internal return signal mechanism.

Bindings hold plain Python objects, not pairs of (value, `LanguageType`). The displayed language type on `print()` is derived from the object's Python class (`int`, `float`, `bool`, `str`), which matches the static rules because only type-checked programs reach this stage.

The built-in `print()` function outputs both value and runtime type, such as `3 : Integer` or `"plc" : String`.

#### ***4.9.1 Implemented Interpreter Scope***

The interpreter work covers all required runtime behaviours. In practical terms, this stage is what gives the language observable behaviour from the user's point of view:

- assignment execution,
- if execution,
- while execution,
- function execution with value parameter passing,
- `print()` execution with immediate output,
- runtime integration through the shared pipeline used by the CLI, UI, and tests.

#### **4.10 Graphical User Interface**

The project includes a desktop UI built with `tkinter`. The interface matches the requested layout: script input appears on the left, while output is shown on the right. The right-hand side contains tabs for execution output, tokens, AST, and inferred types. This allows the user to inspect each stage of the language pipeline from one screen.

## CHAPTER 5

### TESTING AND VERIFICATION

#### 5.1 Test Cases

Testing was performed through both manual execution and automated regression tests. Manual runs were used to inspect the full pipeline from source code to output, while automated tests were added for the semantic and runtime behaviour of the implemented type checker and interpreter.

Table 5.1. Representative Test Cases

| Category      | Program Behaviour                          | Expected Result                     |
|---------------|--------------------------------------------|-------------------------------------|
| Arithmetic    | Evaluate mixed numeric expressions         | Correct value and inferred type     |
| If-then-else  | Choose one branch from a Boolean condition | Only matching branch runs           |
| While-loop    | Print during repeated execution            | One output line per iteration       |
| Functions     | Infer parameters and return type           | Valid call and correct return value |
| Type mismatch | Reassign incompatible type                 | Type error before runtime           |

##### 5.1.1 *Lexer and Symbol Table*

The lexer was verified manually by running the full pipeline and inspecting Stage 1 token output for programs that exercise every token category: integer and float literals, Boolean literals, string literals, all six keywords, two-character comparison operators (`==`, `!=`), single-character operators, and delimiters. Identifier scanning with keyword fallback was confirmed by checking that `true`, `false`, and reserved words are emitted with their keyword token types while non-keyword names produce `IDENTIFIER` tokens.

Error paths were also checked: supplying a bare `!` character raises a `LexerError` with a source position and a message suggesting `!=`, and an unterminated string literal raises a `LexerError` citing the opening line.

The symbol table was verified through the type checker and the Stage 3 output. Duplicate

variable definitions were confirmed to raise a `SymbolTableError` before execution. Scope isolation was verified with nested blocks: a variable defined inside an `if` or `while` body is not visible in the outer scope after the block ends. The `format_table()` output was checked against the expected column layout (name, kind, type, initialisation status, parameters) for both variable and function symbols.

### 5.1.2 *Parser and AST*

The parser was verified manually by running the full pipeline and inspecting Stage 2 AST output. The following cases were checked:

- arithmetic expressions with mixed precedence (e.g. `x + y * 2`) produce a tree where multiplication is a deeper sub-tree than addition, confirming the `_expr/_term/_factor` precedence encoding,
- assignment versus expression statement dispatch: `x = 5`; produces an `Assign` node while `add(1,2)`; produces a `FunctionCall` node at the statement level, confirming the IDENTIFIER+ASSIGN lookahead,
- `if` with and without an `else` clause: the resulting `If` node has a non-null `else_block` only when the clause is present,
- function definitions store only parameter names in the `FunctionDef` node; the AST printer output was inspected to confirm no type annotations are present at this stage,
- string literals in `Literal` nodes have their surrounding double-quotes stripped; `print("hello")` prints `hello : String` at runtime,
- parse errors: missing semicolons, unmatched parentheses, and unexpected tokens all produce `[line X, col Y] ParseError: messages with correct source positions`.

### 5.1.3 *Arithmetic Expressions*

Arithmetic expressions were tested using both integer-only and mixed integer-float programs. These tests verified precedence and numeric promotion. Expressions such as `x + y * 2` confirmed that multiplication is applied before addition.

#### **5.1.4 Boolean Expressions**

Boolean expressions were tested inside `if` and `while` conditions. The checker verified valid comparison operands, and the interpreter used the resulting Boolean values for control flow.

#### **5.1.5 Assignment Statements**

Assignments were tested for both first-use inference and reassignment checks. For example, a variable assigned 5 and later assigned "hello" correctly triggers a type error.

#### **5.1.6 If-Then-Else**

Conditional programs were used to verify that exactly one branch is executed and that non-Boolean conditions are rejected before runtime.

#### **5.1.7 While-Loop**

While-loops were tested with a countdown example. This confirmed that the loop body executes repeatedly and that `print()` emits output progressively on each iteration rather than only once at the end.

#### **5.1.8 Functions**

Function tests covered parameter passing, local execution, and return values. The implementation infers parameter types from the first call and then enforces that signature for later calls.

#### **5.1.9 Print**

The built-in `print()` function was tested with all supported runtime types. The output includes the printed value together with its type, such as `29.5 : Float` and `"plc" : String`.

### **5.2 Type Error Detection**

The checker was verified with invalid programs involving incompatible assignment, invalid arithmetic operands, wrong function arguments, and non-Boolean conditions. In each case the system stopped before execution and reported a source-positioned type error.

### 5.3 Automated Test Suite

Automated regression tests were added in `tests/test_member3_pipeline.py` using the `unittest` framework. The suite currently covers:

- progressive output during while-loop execution,
- assignment type mismatch detection,
- function parameter and return type inference.

The tests can be run using:

```
1 python3 -m unittest discover -s tests -v
```

### 5.4 Error Handling

The language system reports errors at three main stages:

- lexical errors for malformed characters or unterminated strings,
- parse errors for invalid syntax such as missing semicolons,
- type errors for programs that violate static typing rules.

Runtime errors are also reported with source positions whenever execution reaches an invalid state such as an undefined variable reference.

## **CHAPTER 6**

### **CONCLUSION**

This project delivered a working statically-typed programming language with the required primitive types: Integer, Float, Boolean, and String. The language supports arithmetic expressions, equality and inequality checks, assignments, if-then-else statements, while-loops, function abstraction with value parameter passing, return statements, and the built-in `print()` function.

The final system follows a clean compiler-style architecture. The lexer creates tokens, the parser builds the AST, the type checker performs static inference and validation, and the interpreter executes the program. This staged design made it easier to isolate syntax errors, type errors, and runtime behaviour.

One of the main outcomes of the project is the successful implementation of static typing without explicit declarations. Variable types are inferred from assignments, and function signatures are inferred from calls and return statements. Invalid programs are rejected before runtime. In addition, the system includes both a command-line runner and a desktop UI so that scripts can be entered, loaded, executed, and inspected conveniently.

The automated tests added for the semantic and runtime stage pass successfully and confirm correct behaviour for loop execution, assignment mismatch detection, and function type inference. These checks provide a baseline for future extensions.

Future work could include recursive functions, richer data structures, additional operators, and code generation to a lower-level target. However, the current implementation already satisfies the main functional requirements of the assignment.

## REFERENCES

- Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, MA, 2nd edition, 2006.
- Robert Nystrom. *Crafting Interpreters*. Genever Benning, Seattle, WA, 2021.
- Python Software Foundation. Python documentation: dataclasses — data classes. <https://docs.python.org/3/library/dataclasses.html>, 2026a. Accessed: April 2026.
- Python Software Foundation. Python documentation: Tkinter — python interface to tcl/tk. <https://docs.python.org/3/library/tkinter.html>, 2026b. Accessed: April 2026.
- Python Software Foundation. Python documentation: unittest — unit testing framework. <https://docs.python.org/3/library/unittest.html>, 2026c. Accessed: April 2026.

## APPENDIX A: SOURCE CODE AVAILABILITY

The complete source code for this project is available on GitHub:

**<https://github.com/The-Token-Trio/125970-126690-126687-Assignment2-PLC>**

The repository contains the lexer, parser, AST nodes, symbol table, type checker, interpreter, shared execution pipeline, user interface, tests, and the full report source. The project can be run either from the command line through `main.py` or from the desktop UI through `ui.py`.

The main execution commands are:

```
1 python3 main.py
2 python3 ui.py
3 python3 -m unittest discover -s tests -v
```

No external Python dependencies are required for the implementation. The Python side of the project uses only the standard library.

## APPENDIX B: TEAM MEMBERS AND CONTRIBUTIONS

| Name                              | Student ID | Programme |
|-----------------------------------|------------|-----------|
| Aye Khin Khin Hpone (Yolanda Lim) | st125970   | Master's  |
| Applegate Tun Oo                  | st126690   | Master's  |
| Win Htut Naing                    | st126687   | Master's  |

| Name                              | Student ID | Assignment Scope                                                                                                                                                |
|-----------------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Aye Khin Khin Hpone (Yolanda Lim) | st125970   | Static typing rules; type checking; assignment execution; if execution; while execution; function execution; <code>print()</code> execution                     |
| Applegate Tun Oo                  | st126690   | Token definitions; keywords and operators; identifiers and literals; variable/function storage; type storage                                                    |
| Win Htut Naing                    | st126687   | Arithmetic expressions; boolean expressions; assignment statements; if-then-else; while-loop; function definitions; function calls; <code>print()</code> syntax |